操作系统实验报告

实验基本信息

项目	内容
实验名称	Lab 1: 最小可执行内核
小组成员	2411353-李行阳、2412195-邵毅、2412352-王治中
完成日期	2026-10-10

小组分工

成员	负责的练习/模块
2411353-李行阳	完成练习1、2并完成练习2实验报告
2412195-邵毅	完成练习1、2并完成练习1实验报告
2412352-王治中	完成练习1、2并整理实验报告

一、实验目的

实验1主要讲解最小可执行内核和启动流程。我们的内核主要在 QEMU 模拟器上运行，它可以模拟一台 64 位 RISC-V 计算机。为了让我们的内核能够正确对接到 QEMU 模拟器上，需要了解 QEMU 模拟器的启动流程，还需要一些程序内存布局和编译流程（特别是链接）相关知识。

本实验的主要目的是：

1. 理解 RISC-V **从加电到内核运行的启动流程**，弄清复位向量、OpenSBI 固件与内核入口 `kern_entry` 各自的地址与职责。
2. 掌握最小可执行内核的构造方法：用链接脚本描述内存布局、用交叉编译生成内核镜像并对接到 QEMU。
3. 掌握 QEMU + GDB 的**调试方法**（断点、观察点、单步、查看寄存器与内存）。

具体地，本实验需要完成：

- 使用 链接脚本 描述内存布局
- 进行 交叉编译 生成可执行文件，进而生成内核镜像
- 使用 OpenSBI 作为 bootloader 加载内核镜像，并使用 QEMU 进行模拟
- 使用 OpenSBI 提供的服务，在屏幕上格式化打印字符串用于以后调试

二、实验环境

成员	AI 编程工具	底层模型	备注
2411353-李行阳	Claude Code（终端 Agent）	DeepSeek-V4-Pro	
2412195-邵毅	DeepSeek Harness（Web UI）	DeepSeek-flash	
2412352-王治中	Codex（VS Code插件）	DeepSeek-flash	

说明：

- **AI 编程工具**：指具体使用的终端工具、编辑器插件、桌面应用或浏览器界面
- **底层模型**：指该工具使用的大语言模型及版本

三、实验整体逻辑分析

本章节的逻辑主线

本实验的主线，是理解内核从生成到运行的完整过程。这一过程如下：源代码经编译、链接——由链接脚本规定地址布局与入口——再由 QEMU 在 CPU 执行任何指令之前把镜像预置到内存；随后 CPU上电，从固定的复位地址取指，由初始化固件完成硬件初始化、准备好运行环境，再交出控制权，最终到达内核入口，这个链条主要为:

- **阶段一（`0x1000`）**：复位引导代码。CPU 加电只从复位地址开始执行，因此必须存在这样一段不依赖装载即可运行的代码；该地址由 QEMU 的实现规定。它从跳转表读出 OpenSBI 的入口 `0x80000000` 并跳转过去。
- **阶段二（`0x80000000`）**：OpenSBI 固件。内核无法自行完成加载与移交，需要由更底层的固件承担；该地址由复位代码的跳转表约定。OpenSBI 完成初始化后，把控制权交给内核。
- **阶段三（`0x80200000`）**：内核入口 `kern_entry`。内核的装载位置与入口分别由链接脚本的 `BASE_ADDRESS = 0x80200000` 和 `ENTRY(kern_entry)` 规定，必须与 OpenSBI 移交控制权时的约定一致。

因此，本实验的核心是一份关于地址与入口的约定，三个关键点展开：内核**放在哪**，由链接脚本的 `BASE_ADDRESS = 0x80200000` 规定；**由谁放**，镜像由 QEMU 在 CPU 执行任何指令之前预置；**如何开始运行**，控制权到达 `kern_entry` 后先建立内核栈，再进入 C 入口 `kern_init`。这样后便完成内核的加载运行，以下的实验内容可以进一步看到其中的具体过程。

四、实验内容与实现

练习 1：理解内核启动中的程序入口操作

负责人：[2412195-邵毅]

`kern/init/entry.S`：

```
kern_entry:
    la sp, bootstacktop     # 设置内核启动栈
    tail kern_init          # 跳到 C 语言入口

.section .data
    .align PGSHIFT          # 4KB 对齐
bootstack:
    .space KSTACKSIZE       # 2 页 = 8192 字节
bootstacktop:
```

1. `la sp, bootstacktop`

完成的操作（反汇编验证，`obj/kernel.asm`）：

```
80200000: auipc sp,0x3     # sp = pc + 0x3000 = 0x80203000
80200004: mv    sp,sp      # 即 addi sp,sp,0（偏移低 12 位为 0，无操作）
```

`la` 是 PC 相对寻址的伪指令，展开为 `auipc + addi`，把符号 `bootstacktop` 的地址（`0x80203000`，即 `bootstack(0x80201000)` 起 8KB 栈的**顶端**）写入栈指针寄存器 sp(x2)。GDB 单步后 `sp = 0x80203000 <bootstacktop>`，与符号表一致（`bootstack = 0x80201000`，`bootstacktop = 0x80203000`）。

目的：

- 为进入 C 函数建立可用的运行时栈。RISC-V 栈向**低地址**增长，从栈顶 `bootstacktop` 开始，函数序言 `addi sp,sp,-N; sd ra/fp,...` 才能正确压栈；`bootstacktop` 按页对齐，满足 ABI 要求的 16 字节对齐。
- 必须切换，不能沿用上一级固件的栈：加电后 sp 是 OpenSBI 的 `0x8001bd80`，落在 PMP0 保护的固件区 `0x80000000-0x8001ffff`（该 PMP 项无 R/W/X 权限），S 态内核若继续用它压栈会立即触发 store 访问异常。
- 后续 `kern_init → memset / cprintf / va_start` 都要保存 ra、帧指针和局部变量，没有合法栈就无法执行。

2. `tail kern_init`

完成的操作：`tail` 是 RISC-V 调用约定中的"尾调用"伪指令，**不链接返回地址地跳转**到 `kern_init`。目标在 ±1MB 范围内时直接编译成一条跳转（这里是 2 字节的压缩指令）：

```
80200008: a009   j   8020000a <kern_init>
```

GDB 单步结果：`pc: 0x80200008 → 0x8020000a`，`sp` 仍为 `0x80203000`，`ra` 仍为 `0x80000a02`（OpenSBI 留下的返回地址）——即既不写 ra，也不动栈。

目的：`kern_entry` 只是个极小的汇编垫片，真正的 C 语言入口是 `kern_init`（声明为 `__attribute__((noreturn))`，结尾 `while(1);` 永不返回）。既不返回就没有必要 `jal` 保存返回地址、也不必压栈建帧，用 `tail` 直接把控制权"平移"给 C 代码，语义上相当于无条件 `goto kern_init`。

练习2：使用 GDB 验证启动流程

负责人： [2411353-李行阳]

(在QEMU 8.2.2版本下复现，下次实验改为QEMU 4.1.1)

1.复位地址 `0x1000`（加电后最初执行的指令）

连上 QEMU 后，PC 停在复位地址 `0x1000`，与 PPT 所述"PC 被初始化为 0x1000"一致。用 `x/10i $pc` 反汇编当前 PC 开始的 10 条指令：

```
=> 0x1000:  auipc   t0,0x0
   0x1004:  addi    a2,t0,40
   0x1008:  csrr    a0,mhartid
   0x100c:  ld      a1,32(t0)
   0x1010:  ld      t0,24(t0)
   0x1014:  jr      t0
   0x1018:  unimp
   0x101a:  .insn   2, 0x8000
   0x101c:  unimp
   0x101e:  unimp
```

观察到的两点：

- 前 6 条指令（`0x1000` ~ `0x1014`）是真正的代码；`0x1014` 之后的 `unimp`/`.insn` 并不是指令，而是**跳转表数据**（`0x1018` 存 `0x80000000`、`0x1020` 存 `0x87e00000`、`0x1028` 存 magic "OSBI"），只是被 `x/10i` 的 `i`（按指令解码）强行当成指令翻译。
- 这段代码**既不是 OpenSBI 也不是我们编译的内核**，而是 QEMU 内置的一小段复位引导代码（reset vector），和 `0x80000000` 一样写死在 QEMU 中。它整体只做一件事："读出跳转表里的 OpenSBI 入口并跳过去"。

2.跳转到 OpenSBI 入口 `0x80000000`

用 `si 6` 单步执行这 6 条指令，PC 正好跳到 `0x80000000`——即 OpenSBI（M 模式固件）的入口，这对应"第二阶段"，也是跳转到内核 `0x80200000` 之前的重要一步。再用 `x/10i $pc` 查看：

```
=> 0x80000000:  add s0,a0,zero
   0x80000004:  add s1,a1,zero
   0x80000008:  add s2,a2,zero
   0x8000000c:  jal 0x80000580
   ...
```

观察到：OpenSBI 入口先把上一阶段传来的三个参数（`a0`=hartid、`a1`=设备树地址、`a2`=fw_dynamic 信息指针）保存到 callee-saved 寄存器 `s0/s1/s2`，随后 `jal 0x80000580` 进入主初始化。**从这里开始，OpenSBI 才真正进行硬件主初始化、并把内核加载/移交到** `0x80200000` ——这是三个阶段的"第二棒"。

3.验证内核入口 0x80200000

同时设置观察点 `watch *0x80200000`（观察内核**加载**）和断点 `b *0x80200000`（观察内核**执行**），`c` 继续执行：

```
Breakpoint 2, kern_entry () at kern/init/entry.S:7
```

命中的是**断点**（`kern_entry`），而非观察点。用 `x/10i $pc` 查看内核入口指令：

```
=> 0x80200000 <kern_entry>:   auipc sp,0x3
   0x80200004 <kern_entry+4>: mv    sp,sp
   0x80200008 <kern_entry+8>: j     0x8020000a <kern_init>
   ...
```

观察到：内核第一条指令是 `kern_entry`（`kern/init/entry.S`），设置内核栈指针后尾跳转到 C 入口 `kern_init`——即"第三棒"，操作系统的真正起点。

关键观察：`watch *0x80200000` 观察点**始终没有触发**，只有断点命中了。这不是操作失误，而是揭示了一个机制：GDB 的 `watch` 只能捕捉 **CPU 通过执行 store 指令产生的写入**；而在本实验的 `-kernel` 加载方式下，内核是 **QEMU 在 CPU 执行任何指令之前直接把 ELF 写进** `0x80200000` **的**，并不经过 CPU 的 store 指令，所以观察点抓不到这次"加载"。而实验指导书里"用 watch 观察内核加载瞬间"的写法，其实更契合**真实机器**的启动逻辑——在真实机器上，Bootloader（如 U-Boot）必须用指令从存储介质读取内核并显式搬运到 `0x80200000`，此时 `watch` 才能准确命中。而在本实验的 QEMU 环境下，这一预期与实际不符的现象，反而更真实地揭示了 QEMU 在加电前预加载内核的机制。

4.问题解答汇总

问题1：RISC-V 硬件加电后最初执行的几条指令位于什么地址?

位于**复位地址** `0x1000`，具体是 `0x1000` ~ `0x1014` 共 6 条指令。

依据（实验观察）：GDB 连上 QEMU 后 PC 停在 `0x1000`；`x/10i $pc` 反汇编出的前 6 条指令是真正的代码（`auipc`/`addi`/`csrr`/`ld`/`ld`/`jr`），`0x1014` 之后则是跳转表数据（被 `x/i` 误当指令解码）。

问题2：它们主要完成了哪些功能?

这 6 条指令是**最精简的"启动引导代码"，不负责复杂的硬件初始化**，只完成"找到 OpenSBI 并跳过去"这一个功能，具体分四步：

1. **取得自身基址**：`auipc t0,0x0` 把当前 PC 存入 `t0`（`t0 = 0x1000`），作为后续读数据的基准地址；
2. **准备传递给 OpenSBI 的参数**：`csrr a0,mhartid` 读出硬件线程号（第一个参数），`addi a2,t0,40` 算出 fw_dynamic 信息结构指针，`ld a1,32(t0)` 读出设备树地址 `0x87e00000`；
3. **从跳转表读出 OpenSBI 入口**：`ld t0,24(t0)` 从 `0x1018` 读出 `0x80000000`；
4. **跳转**：`jr t0` 跳到 `0x80000000`，把控制权交给 OpenSBI。

概括地说，这 6 条指令是"启动接力棒的第一棒"——它们只负责找到并跳转到 OpenSBI，而真正的硬件初始化、内核加载（到 `0x80200000`）以及特权级切换，都由 OpenSBI（第二阶段）完成。

五、测试与验证

练习2:

1. **make qemu**

```
lixingyang@LAPTOP-BLHGNE83:~/project2/os-riscv/code$ make qemu

OpenSBI v1.3
   ____                    _____ ____ _____
  / __ \                  / ____|  _ \_   _|
 | |  | |_ __   ___ _ __ | (___ | |_) || |
 | |  | | '_ \ / _ \ '_ \ \___ \|  _ < | |
 | |__| | |_) |  __/ | | |____) | |_) || |_
  \____/| .__/ \___|_| |_|_____/|___/_____|
        | |
        |_|

Platform Name             : riscv-virtio,qemu
Platform Features         : medeleg
Platform HART Count       : 1
Platform IPI Device       : aclint-mswi
Platform Timer Device     : aclint-mtimer @ 10000000Hz
Platform Console Device   : uart8250
Platform HSM Device       : ---
Platform PMU Device       : ---
Platform Reboot Device    : sifive_test
Platform Shutdown Device  : sifive_test
Platform Suspend Device   : ---
Platform CPPC Device      : ---
Firmware Base             : 0x80000000
Firmware Size             : 322 KB
Firmware RW Offset        : 0x40000
Firmware RW Size          : 66 KB
Firmware Heap Offset      : 0x48000
Firmware Heap Size        : 34 KB (total), 2 KB (reserved), 9 KB (used), 22 KB (free)
Firmware Scratch Size     : 4096 B (total), 760 B (used), 3336 B (free)
Runtime SBI Version       : 1.0

Domain0 Name              : root
```

2. **复位地址** `0x1000`

```
lixingyang@LAPTOP-BLHGNE83:~/project2/os-riscv/code$ gdb-multiarch -q bin/ker
nel
Reading symbols from bin/kernel...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
0x0000000000001000 in ?? ()
(gdb) x/10i $pc
=> 0x1000:      auipc   t0,0x0
   0x1004:      addi    a2,t0,40
   0x1008:      csrr    a0,mhartid
   0x100c:      ld      a1,32(t0)
   0x1010:      ld      t0,24(t0)
   0x1014:      jr      t0
   0x1018:      unimp
   0x101a:      .insn   2, 0x8000
   0x101c:      unimp
   0x101e:      unimp
```

3. **跳转到 OpenSBI 入口** `0x80000000`

```
(gdb) si 6
0x0000000080000000 in ?? ()
(gdb) x/10i $pc
=> 0x80000000:  add     s0,a0,zero
   0x80000004:  add     s1,a1,zero
   0x80000008:  add     s2,a2,zero
   0x8000000c:  jal     0x80000580
   0x80000010:  add     a6,a0,zero
   0x80000014:  add     a0,s0,zero
   0x80000018:  add     a1,s1,zero
   0x8000001c:  add     a2,s2,zero
   0x80000020:  li      a7,-1
   0x80000022:  beq     a6,a7,0x8000002a
```

4. **验证内核入口** `0x80200000`

```
(gdb) watch *0x80200000
Hardware watchpoint 1: *0x80200000
(gdb) b *0x80200000
Note: breakpoint 1 also set at pc 0x80200000.
Breakpoint 2 at 0x80200000: file kern/init/entry.S, line 7.
(gdb) c
Continuing.

Breakpoint 2, kern_entry () at kern/init/entry.S:7
7           la sp, bootstacktop
(gdb) x/10i $pc
=> 0x80200000 <kern_entry>:     auipc   sp,0x3
   0x80200004 <kern_entry+4>:   mv      sp,sp
   0x80200008 <kern_entry+8>:   j       0x8020000a <kern_init>
   0x8020000a <kern_init>:      auipc   a0,0x3
   0x8020000e <kern_init+4>:    addi    a0,a0,-2
   0x80200012 <kern_init+8>:    auipc   a2,0x3
   0x80200016 <kern_init+12>:   addi    a2,a2,-10
   0x8020001a <kern_init+16>:   addi    sp,sp,-16
   0x8020001c <kern_init+18>:   li      a1,0
   0x8020001e <kern_init+20>:   sub     a2,a2,a0
(gdb) c
Continuing.
(gdb)
```

六、实验总结与收获

对操作系统的理解

1. **启动流程**：内核不是自己跑起来的，而是一层层传递执行过来的——复位向量（`0x1000`，M 态）→ OpenSBI（`0x80000000`，M 态）→ 内核入口

`kern_entry`（`0x80200000`，S 态）。
2. **内核镜像的构造**：链接脚本决定内核必须落在 `0x80200000`，而"装进内存"是 QEMU 在 CPU 开始执行之前完成的。
3. **调试方法**：QEMU + GDB 让我们能直接调试内核：用`p/x $pc` 查看pc地址，用 `x/10i $pc` 查看pc中对应的指令，用断点跳过大量固件代码。`watch *0x80200000` 始终不触发反而是最大收获——它让我们知道内核镜像是谁装进去的。
4. **固件意义**：应用程序的运行依赖操作系统为其准备环境；而操作系统本身也是软件，同样需要固件为其准备运行环境，这正是固件存在的意义。固件依赖于硬件复位行为（如CPU上电后从固定地址取指）作为起点。到了硬件这一层，启动不再依赖任何软件。

AI 协作开发的经验

1. **AI 给方向，结论要用调试证据核实**：指导书建议用 `watch *0x80200000` 观察内核加载，实测该观察点始终不触发，追下去才弄清镜像由 QEMU 在 CPU 执行之前预置、不经过 store 指令。
2. **对提示词工程的理解更深**：把要求讲明白（做什么、能依赖什么、要交付什么、规格是什么），比笼统的提示词更容易拿到可用结果，也更容易把新知识连接到已有的知识框架上；另外，四段式提示词框架规范了我们提示词的书写。